From inputs to predictions

This section shows how a model generates predictions step by step from an input sentence.

Step 1: Tokenize the input and pass it to the model

First, we load the tokenizer and the model:

from transformers import AutoTokenizer, AutoModelForTokenClassification

model_checkpoint = "dbmdz/bert-large-cased-finetuned-conll03-english"

tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)

model = AutoModelForTokenClassification.from_pretrained(model_checkpoint)

example = "My name is Sylvain and I work at Hugging Face in Brooklyn."

inputs = tokenizer(example, return_tensors="pt")

outputs = model(**inputs)

What is happening here:

- `AutoTokenizer` splits the sentence into tokens
- `AutoModelForTokenClassification` predicts labels for those tokens
- `return_tensors="pt"` means the output will be returned as PyTorch tensors

Step 2: Understand the model output shape

print(inputs["input_ids"].shape)

print(outputs.logits.shape)

Output:

torch.Size([1, 19])

torch.Size([1, 19, 9])

This means:

- batch size = 1
- total tokens = 19
- the model gives 9 label scores for each token

So the output shape of the model is:

`1 x 19 x 9`

Meaning:

- 1 sentence
- 19 tokens
- 9 possible classes for each token

Step 3: Get predictions from logits

The model does not directly output the final label. It first produces raw scores, called **logits**.

Then `softmax` is used to convert them into probabilities, and `argmax` is used to select the most likely label.

import torch

probabilities = torch.nn.functional.softmax(outputs.logits, dim=-1)[0].tolist()

predictions = outputs.logits.argmax(dim=-1)[0].tolist()

print(predictions)

Output:

[0, 0, 0, 0, 4, 4, 4, 4, 0, 0, 0, 0, 6, 6, 6, 0, 8, 0, 0]

Here:

- `softmax()` converts raw logits into probabilities
- `argmax()` takes the label index with the highest score

Step 4: Understand the meaning of the label indices

model.config.id2label

Output:

{

0: 'O',

1: 'B-MISC',

2: 'I-MISC',

3: 'B-PER',

4: 'I-PER',

5: 'B-ORG',

6: 'I-ORG',

7: 'B-LOC',

8: 'I-LOC'

}

Here:

- `O` means the token does not belong to any entity
- `B-XXX` means the beginning of an entity
- `I-XXX` means the inside part of an entity

Examples:

- `B-PER` = beginning of a person entity

- `I-PER` = inside a person entity
- `B-ORG` = beginning of an organization entity
- `I-ORG` = part of an organization entity
- `B-LOC` = beginning of a location entity
- `I-LOC` = part of a location entity

Step 5: Why did the token `S` also get `I-PER`?

At first, we might think:

- `S` should be `B-PER`
- `##yl`, `##va`, `##in` should be `I-PER`

But the model gave `I-PER` even to `S`.

This is not completely wrong, because there are two entity-labeling formats:

IOB2

Here, `B-` is used whenever an entity starts.

IOB1

Here, `B-` is mainly used when two entities of the same type are next to each other and need to be separated.

The dataset used to fine-tune this model follows the **IOB1** format. That is why the token `S` can also receive the label `I-PER`.

Step 6: Extract all entity tokens except `O`

```
results = []

tokens = inputs.tokens()



for idx, pred in enumerate(predictions):

    label = model.config.id2label[pred]

    if label != "O":
```

```
        results.append(
            {"entity": label, "score": probabilities[idx][pred], "word": tokens[idx]}
        )

print(results)
```

The output will look like this:

```
[
 {'entity': 'I-PER', 'score': 0.9993828, 'word': 'S'},
 {'entity': 'I-PER', 'score': 0.99815476, 'word': '##yl'},
 {'entity': 'I-PER', 'score': 0.99590725, 'word': '##va'},
 {'entity': 'I-PER', 'score': 0.9992327, 'word': '##in'},
 {'entity': 'I-ORG', 'score': 0.97389334, 'word': 'Hu'},
 {'entity': 'I-ORG', 'score': 0.976115, 'word': '##gging'},
 {'entity': 'I-ORG', 'score': 0.98879766, 'word': 'Face'},
 {'entity': 'I-LOC', 'score': 0.99321055, 'word': 'Brooklyn'}
]
```

Here we:

- looked at the predictions for all tokens
- kept only the ones whose label was not 0
- stored their entity label, score, and token

Step 7: Use offset mapping to find start and end positions

One thing was missing from the previous result:

- where exactly each token appeared in the original sentence

For that, we use `return_offsets_mapping=True`.

inputs_with_offsets = tokenizer(example, return_offsets_mapping=True)

inputs_with_offsets["offset_mapping"]

Output:

[(0, 0), (0, 2), (3, 7), (8, 10), (11, 12), (12, 14), (14, 16), (16, 18), (19, 22), (23, 24), (25, 29), (30, 32),

(33, 35), (35, 40), (41, 45), (46, 48), (49, 57), (57, 58), (0, 0)]

Each tuple shows the character range in the original text that corresponds to a token.

For example:

- `(12, 14)` means from index 12 to 14 in the text

`(0, 0)` is for special tokens such as `[CLS]` and `[SEP]`.

Example:

example[12:14]

Output:

yl

That means the token `##yl` matches the substring `yl` in the original text.

Step 8: Add start and end positions to the final result

results = []

inputs_with_offsets = tokenizer(example, return_offsets_mapping=True)

tokens = inputs_with_offsets.tokens()

```
offsets = inputs_with_offsets["offset_mapping"]


for idx, pred in enumerate(predictions):
    label = model.config.id2label[pred]
    if label != "O":
        start, end = offsets[idx]
        results.append(
            {
                "entity": label,
                "score": probabilities[idx][pred],
                "word": tokens[idx],
                "start": start,
                "end": end,
            }
        )

print(results)
```

Now the output will be:

```
[
 {'entity': 'I-PER', 'score': 0.9993828, 'word': 'S', 'start': 11, 'end': 12},
 {'entity': 'I-PER', 'score': 0.99815476, 'word': '##yl', 'start': 12, 'end': 14},
 {'entity': 'I-PER', 'score': 0.99590725, 'word': '##va', 'start': 14, 'end': 16},
 {'entity': 'I-PER', 'score': 0.9992327, 'word': '##in', 'start': 16, 'end': 18},
```

{'entity': 'I-ORG', 'score': 0.97389334, 'word': 'Hu', 'start': 33, 'end': 35},

{'entity': 'I-ORG', 'score': 0.976115, 'word': '##gging', 'start': 35, 'end': 40},

{'entity': 'I-ORG', 'score': 0.98879766, 'word': 'Face', 'start': 41, 'end': 45},

{'entity': 'I-LOC', 'score': 0.99321055, 'word': 'Brooklyn', 'start': 49, 'end': 57}

]

This is the same kind of result as the first pipeline output.